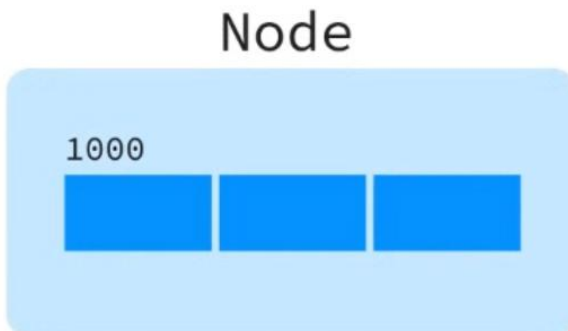
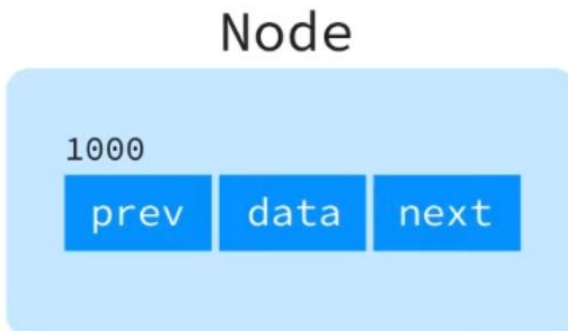
DATA STRUCTURES

Doubly Linked List

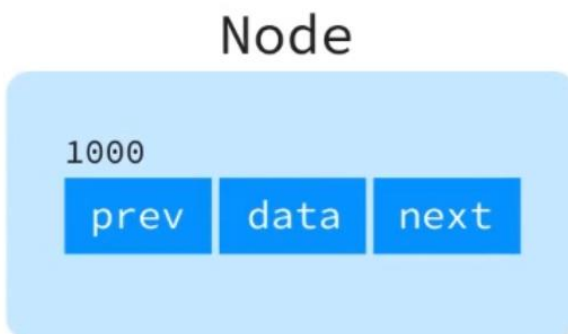
log2base2.com



log2base2.com

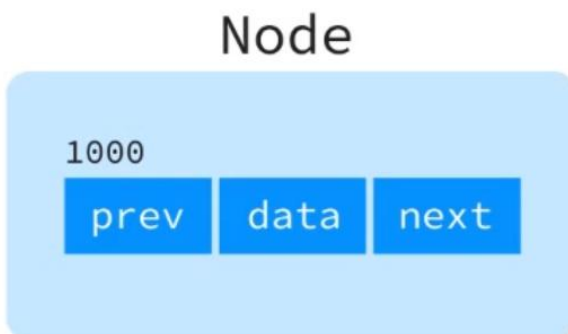


log2base2.com



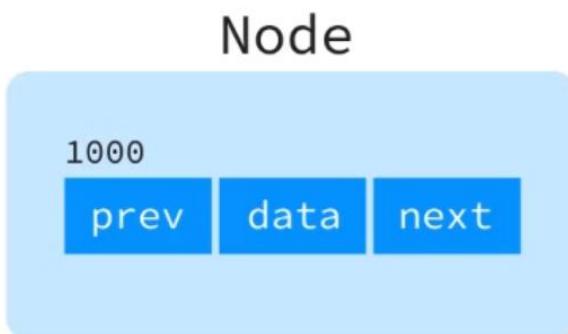

```
struct node
{
   struct node *prev;
   int data;
   struct node * next;
};
```

log2base2.com



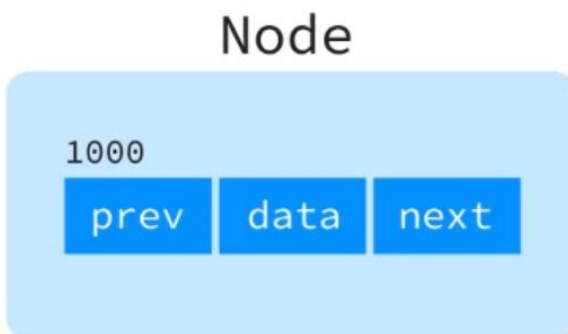

```
struct node
{
    struct node *prev;
    int data;
    struct node * next;
};
```

log2base2.com




```
struct node
{
    struct node *prev;
    int data;
    struct node * next;
};
```

log2base2.com




```
struct node
{
    struct node *prev;
    int data;
    struct node * next;
};
```

log2base2.com

Example

log2base2.com

log2base2.com

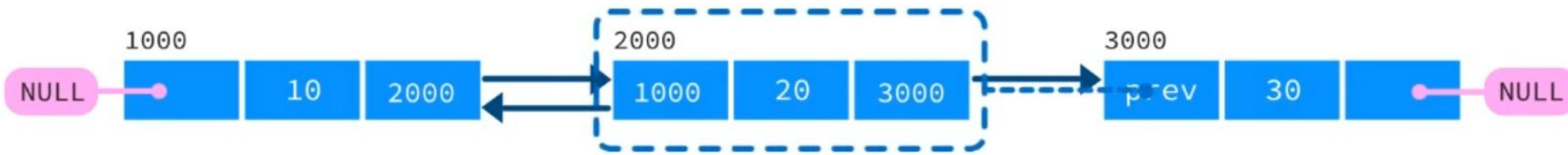
log2base2.com

log2base2.com

log2base2.com

log2base2.com

log2base2.com



log2base2.com

log2base2.com

DATA STRUCTURES

Linked Lists

log2base2.com

LINKED LIST

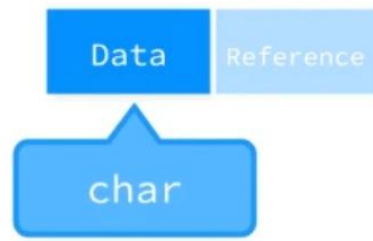
LINEAR DATA STRUCTURE

log2base2.com

LINKED LIST

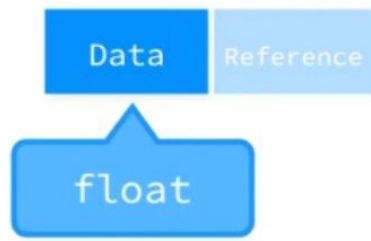
log2base2.com

LINKED LIST



log2base2.com

LINKED LIST



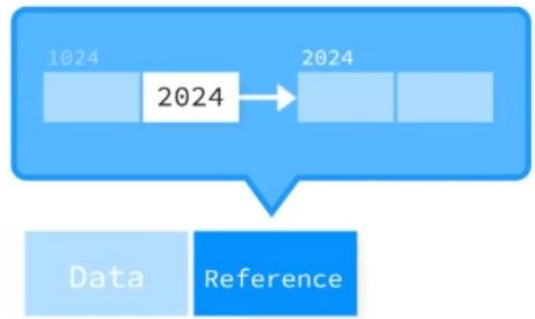
log2base2.com

LINKED LIST

```
int data;
```

log2base2.com

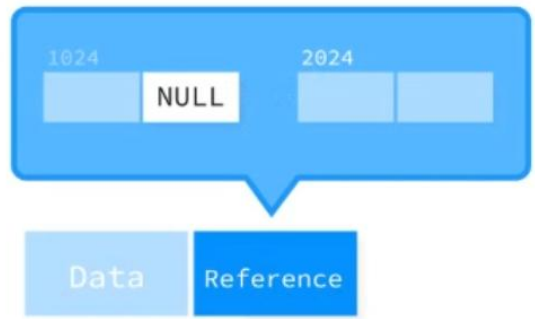
LINKED LIST




```
int data;
```

log2base2.com

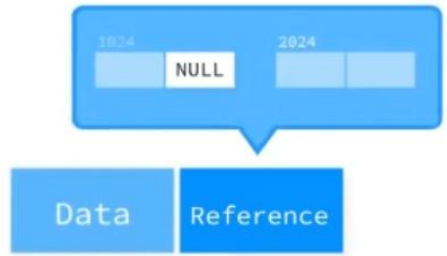
LINKED LIST




```
int data;
str
```

log2base2.com

LINKED LIST




```
int data;
struct node *next;
```

log2base2.com

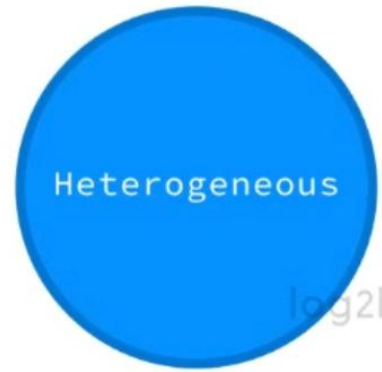
LINKED LIST

```
int data;
struct node *next;
```

log2base2.com

LINKED LIST

```
int data;
struct node *next;
```



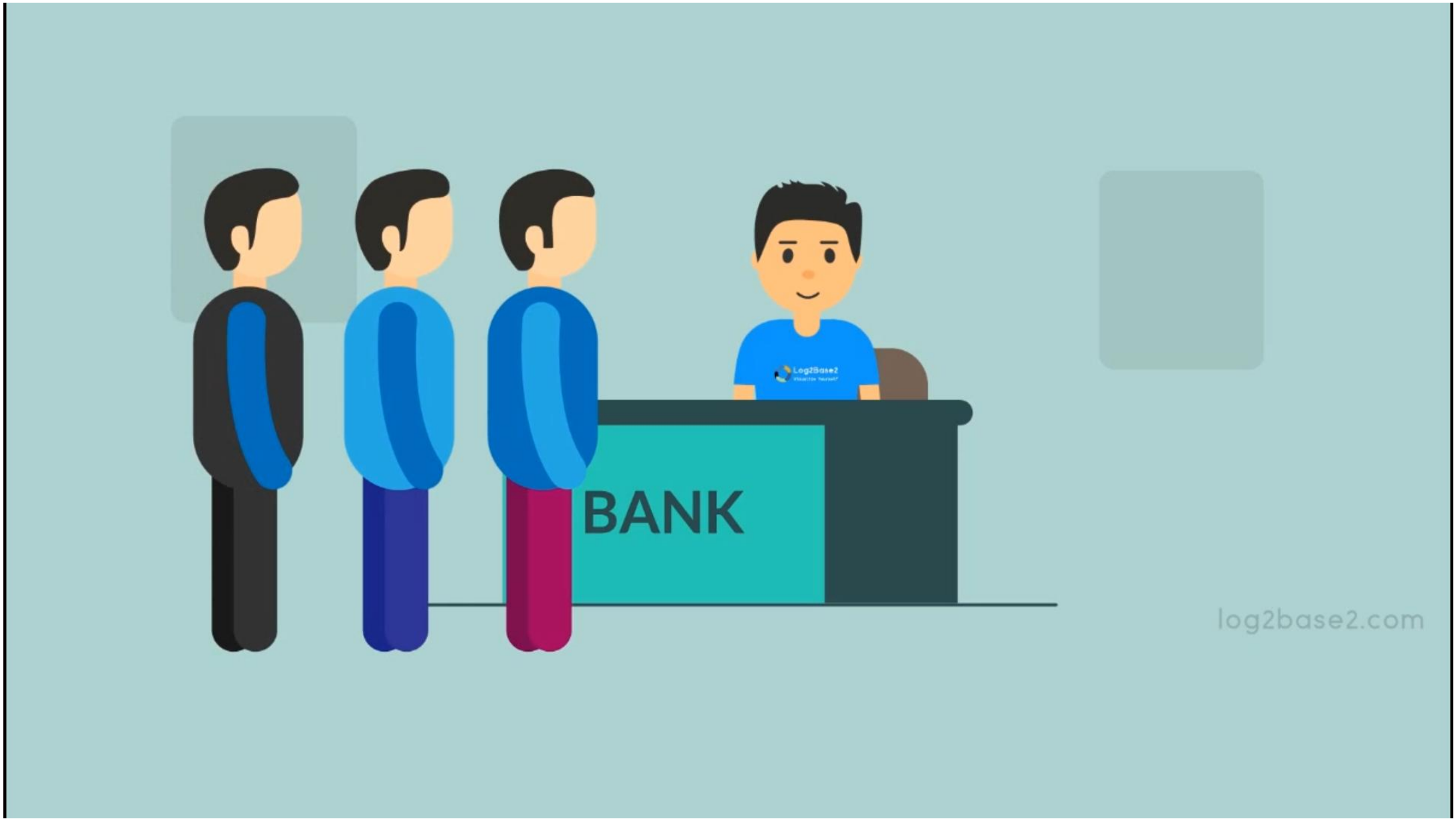
log2base2.com

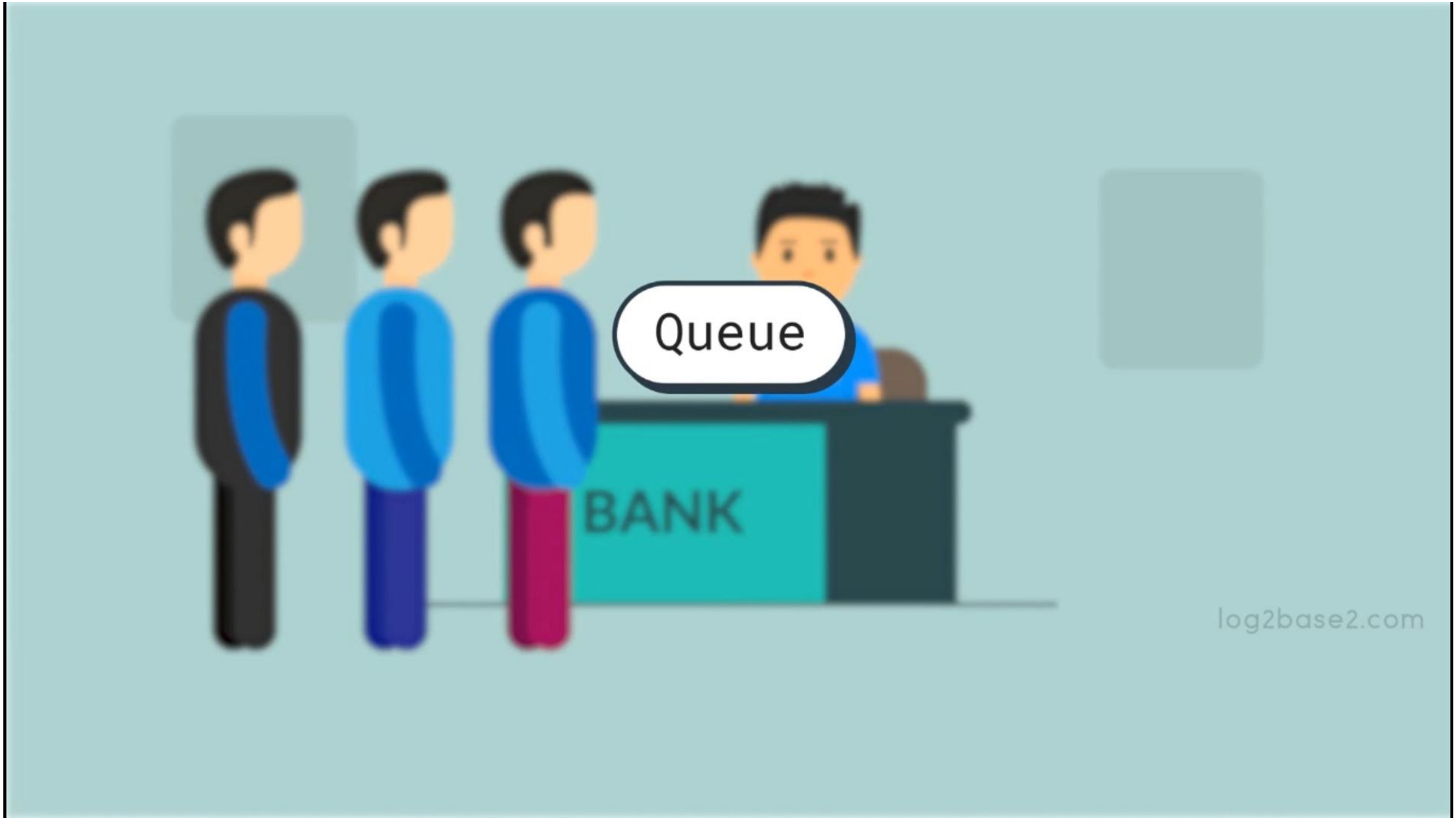
DATA STRUCTURES

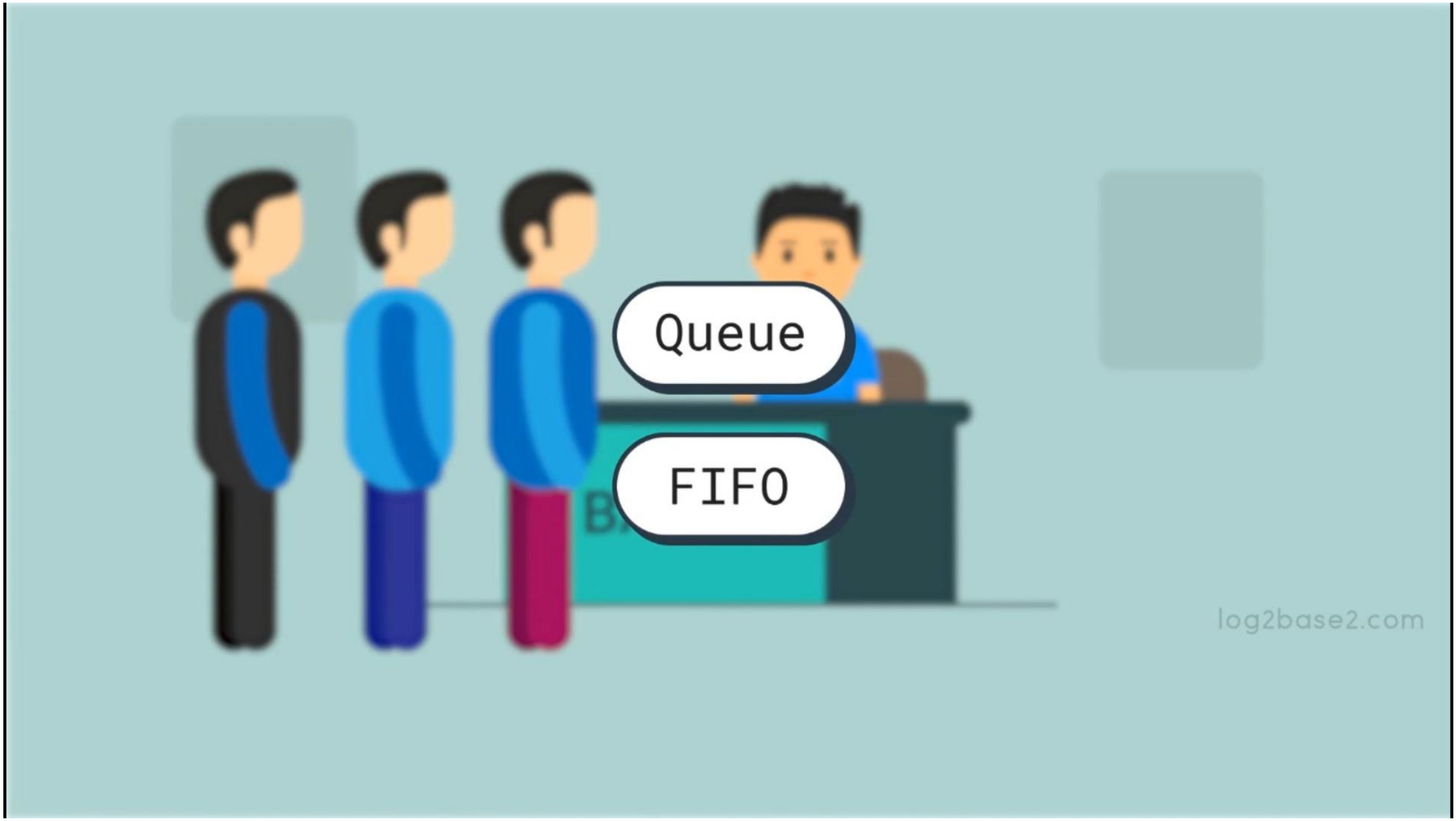
Queue

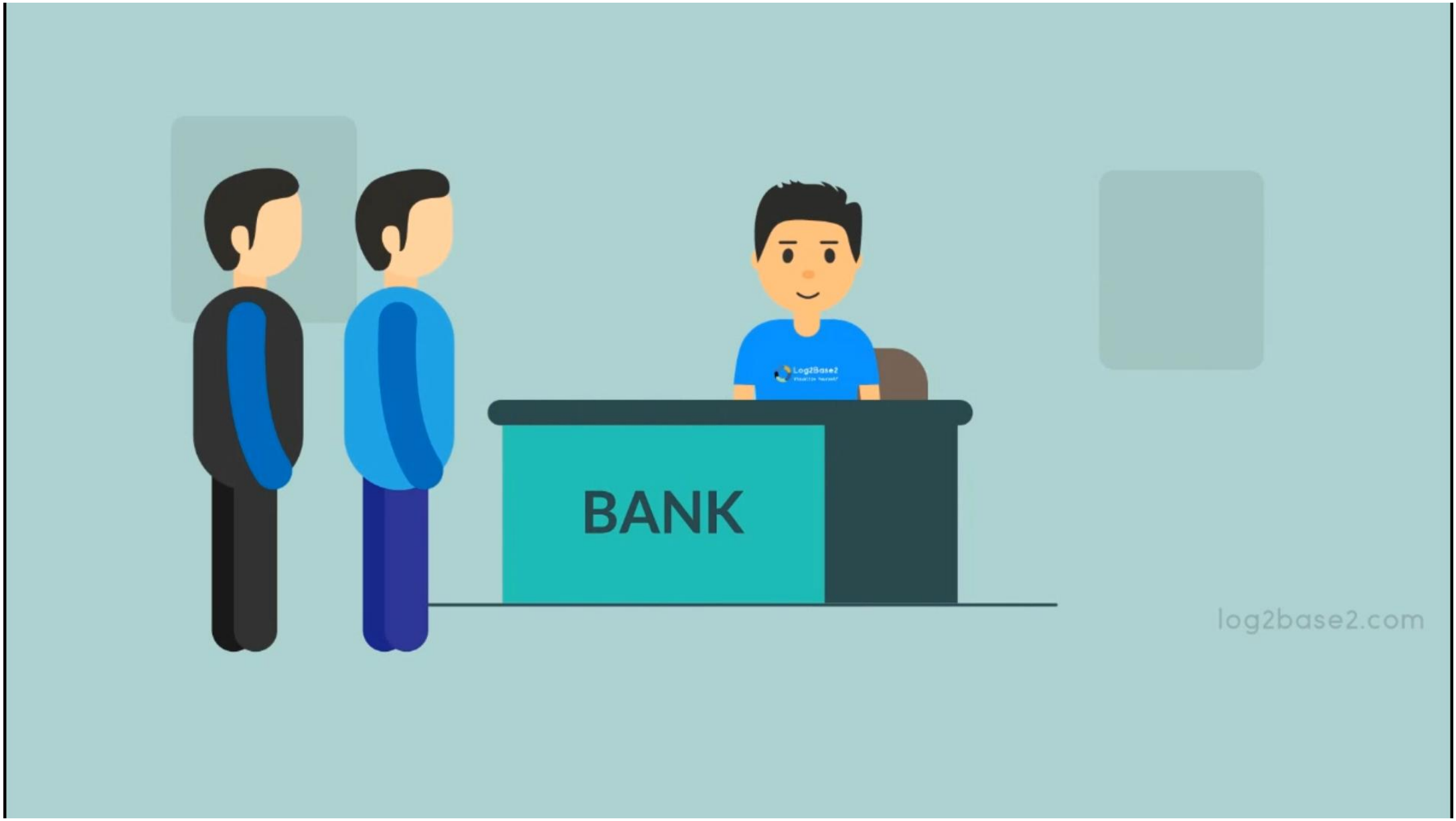
log2base2.com

log2base2.com


BANK
log2base2.com


Queue
BANK
log2base2.com


Queue
FIFO
log2base2.com

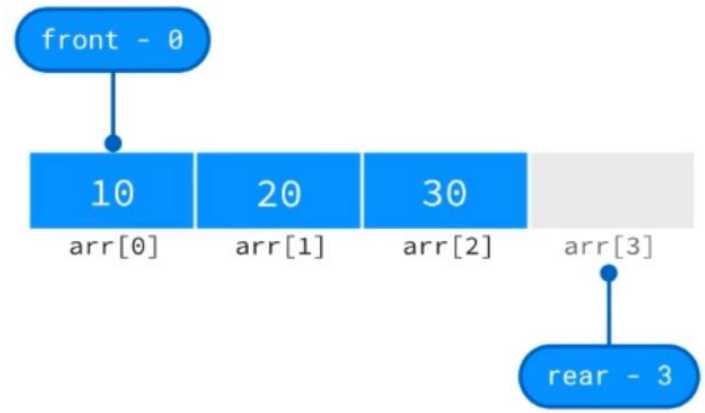

BANK
log2base2.com

```
#define size 3
int arr[size];

int front  = 0;
int rear   = 0;

void enqueue(int val)
{        3  ==  3
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}
```

Val - 40



log2base2.com

```
#define size 3
int arr[size];

int front  = 0;
int rear   = 0;

void enqueue(int val)
{        3  ==  3
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}
```

Val - 40

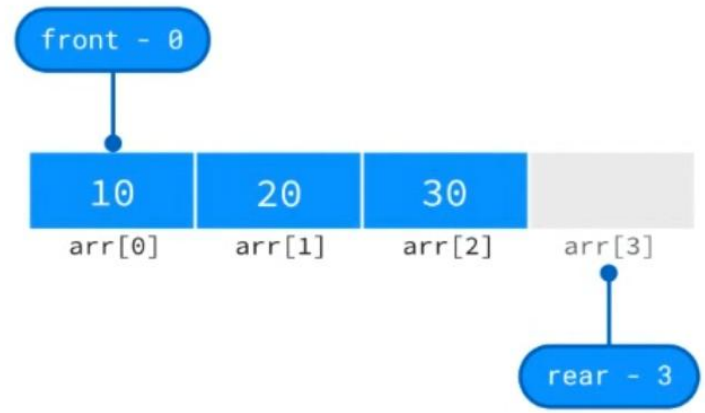
log2base2.com

```c
#define size 3
int arr[size];

int front  = 0;
int rear   = 0;

void enqueue(int val)
{        3  ==  3
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}
```

Val - 40



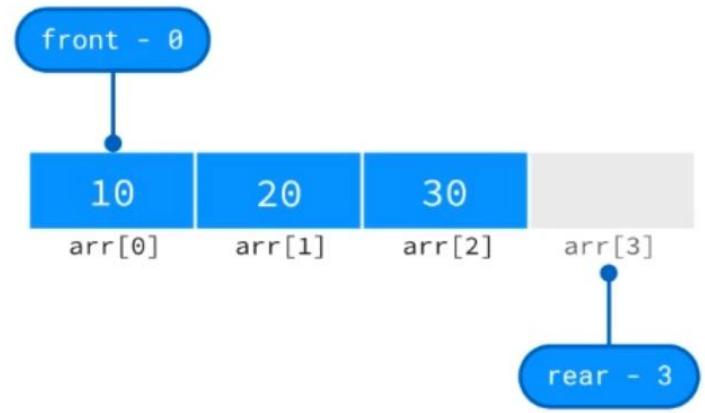
log2base2.com

```
int front  = 0;
int rear   = 0;

void enqueue(int val)
{
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}

void dequeue()
{
   if(front == rear)
       printf("Queue is Empty.\n");
   else
   {
       printf("Dequeued element = %d\n",arr[front]);
       front++;
   }
}
```

Dequeued element = 10

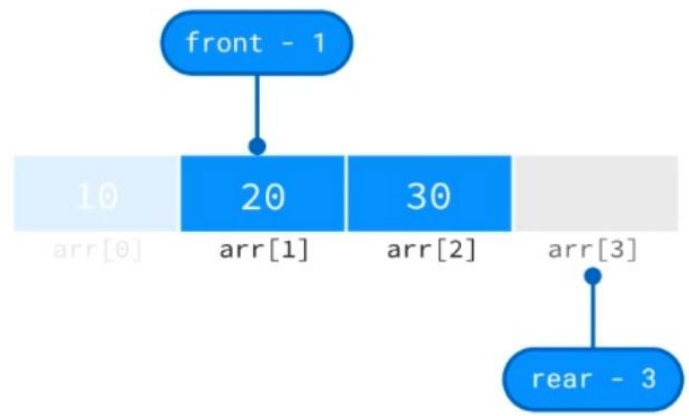


log2base2.com

```
int front  = 0;
int rear   = 0;

void enqueue(int val)
{
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}

void dequeue()
{
   if(front == rear)
       printf("Queue is Empty.\n");
   else
   {
       printf("Dequeued element = %d\n",arr[front]);
       front++;
   }
}
```



log2base2.com

```
int front  = 0;
int rear   = 0;

void enqueue(int val)
{
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}

void dequeue()
{
   if(front == rear)
       printf("Queue is Empty.\n");
   else
   {
       printf("Dequeued element = %d\n",arr[front]);
       front++;
   }
}
```

log2base2.com

```
int front  = 0;
int rear   = 0;

void enqueue(int val)
{
    if(rear == size)
        printf("Queue is Full\n");
    else
    {
        arr[rear] = val;
        rear++;
    }
}

void dequeue()
{
    if(front == rear)
        printf("Queue is Empty.\n");
    else
    {
        printf("Dequeued element = %d\n",arr[front]);
        front++;
    }
}
```

log2base2.com

log2base2.com

log2base2.com

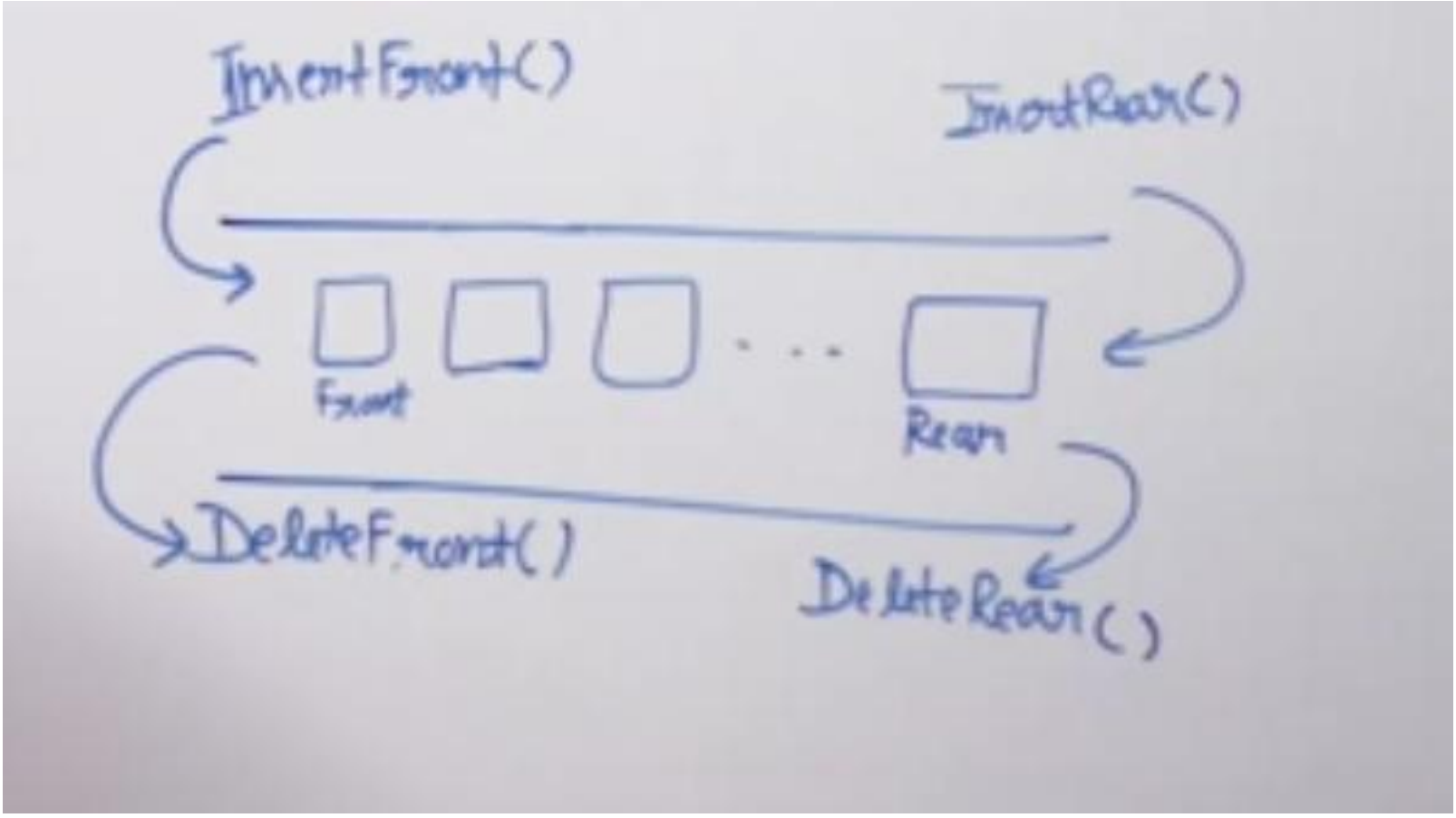
log2base2.com

log2base2.com

log2base2.com

log2base2.com

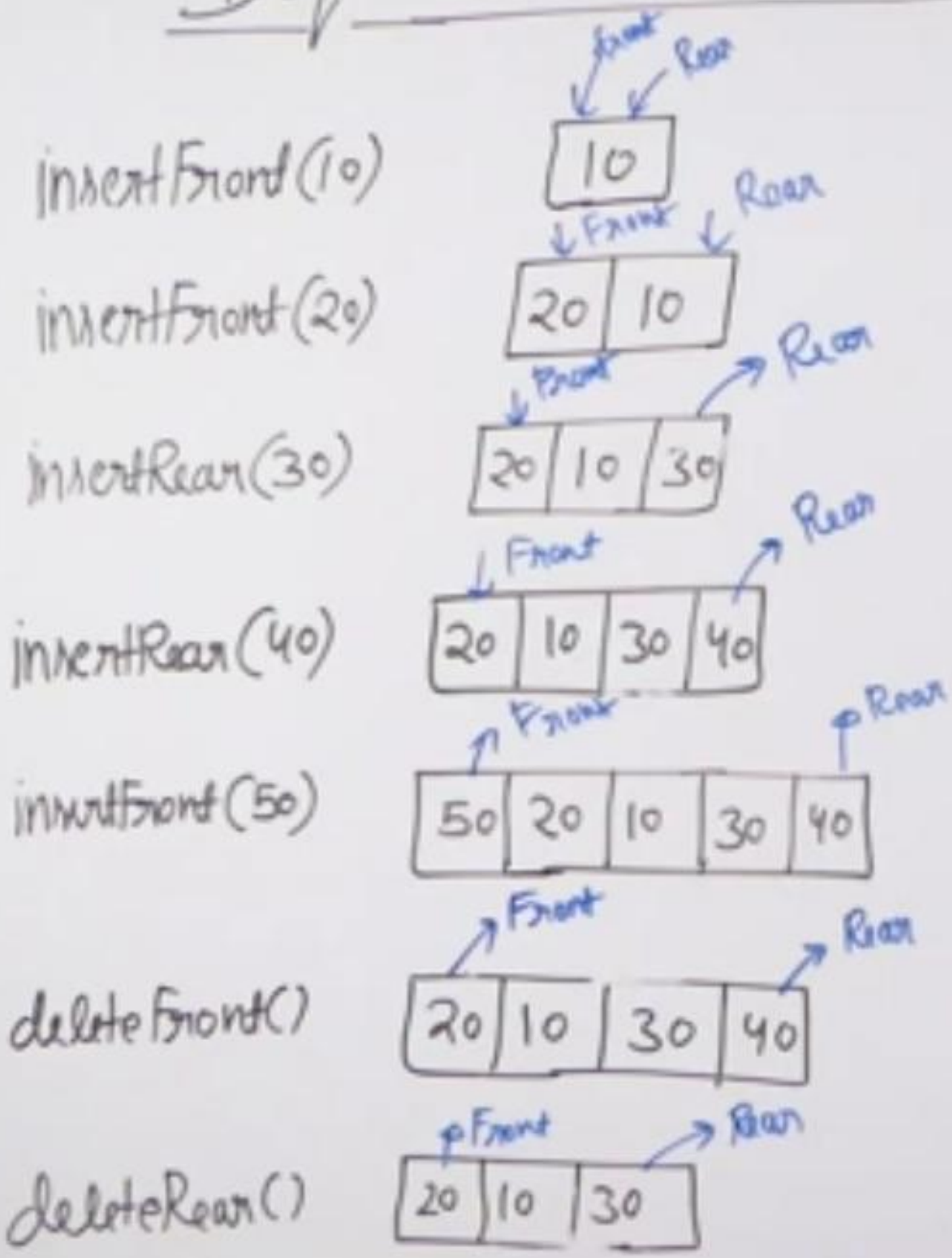
log2base2.com


InsertFront()
InsertRear()
Front
Rear
DeleteFront()
DeleteRear()

Deque Data Structure

insertFront(10)	10
insertFront(20)	20 | 10
insertRear(30)	20 | 10 | 30
insertRear(40)	20 | 10 | 30 | 40
insertFront(50)	50 | 20 | 10 | 30 | 40
deleteFront()	20 | 10 | 30 | 40
deleteRear()	20 | 10 | 30

Deque Data Structure

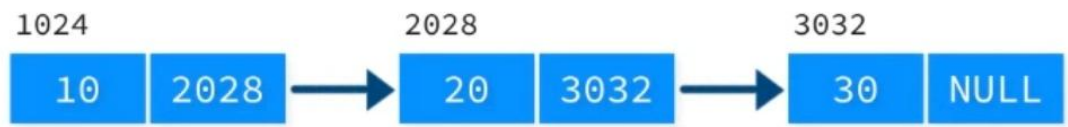


DATA STRUCTURES

Binary Tree Basics

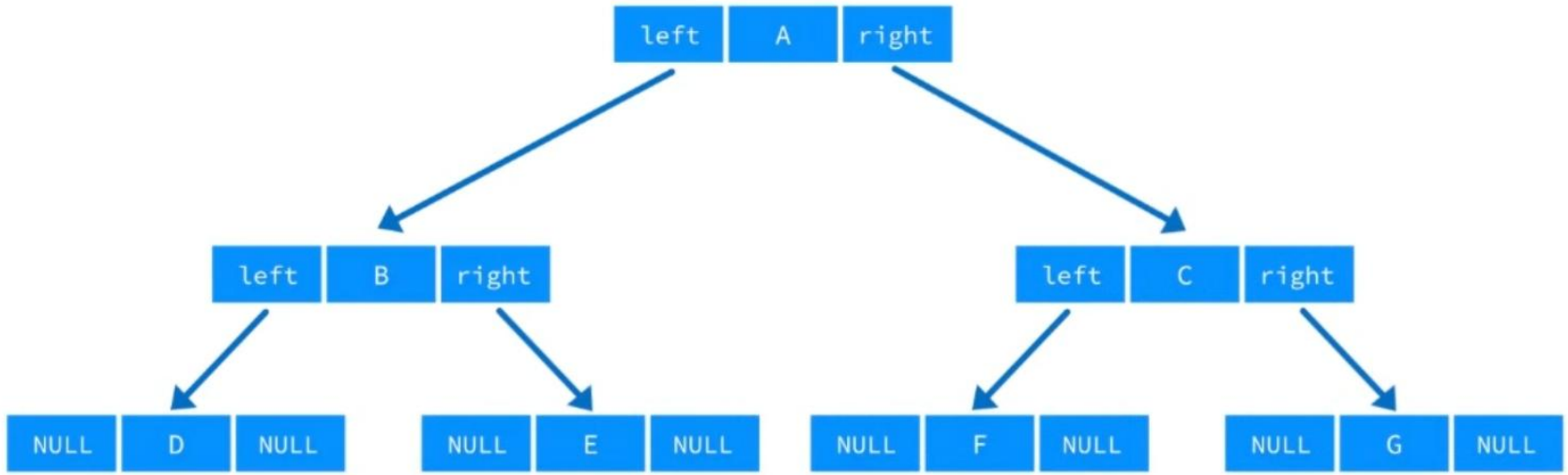
log2base2.com

Linked List



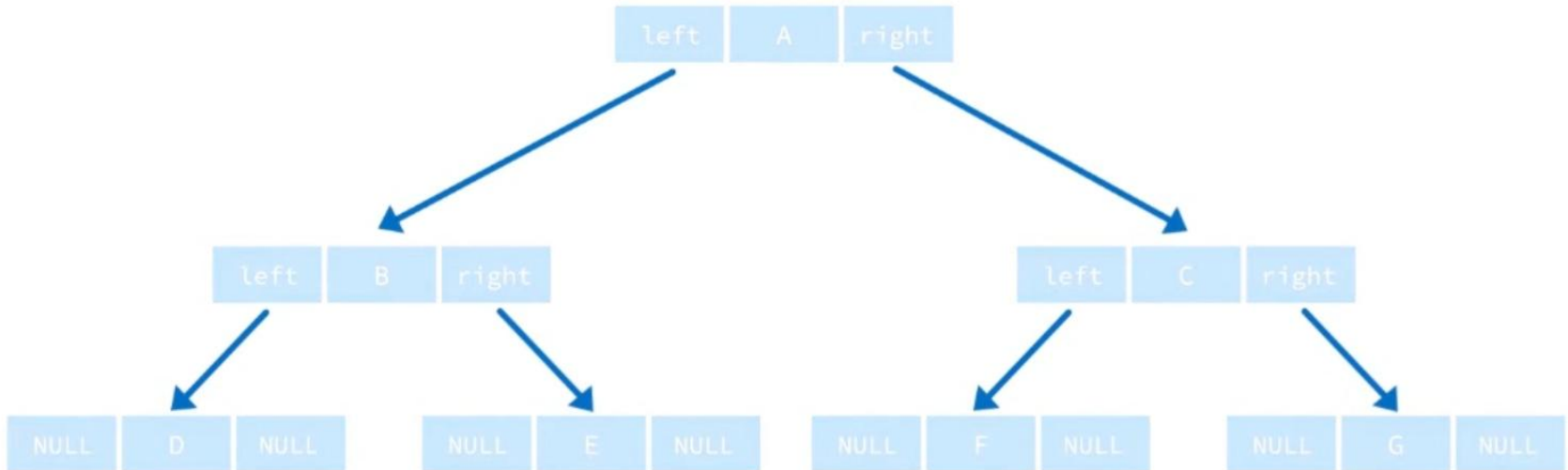
log2base2.com

Binary Tree



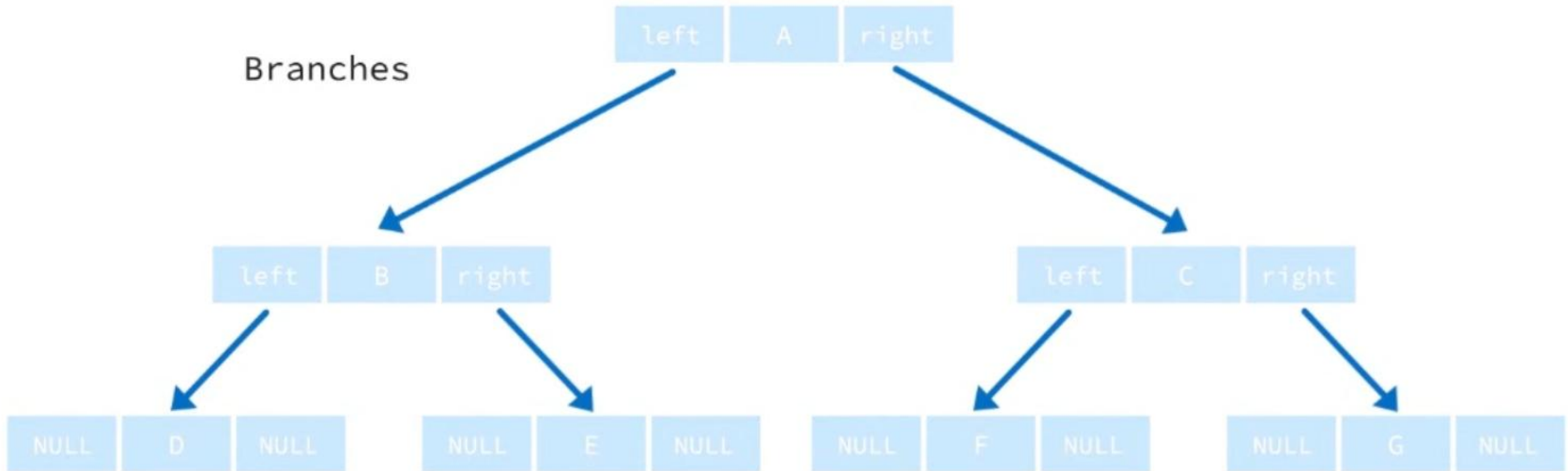
log2base2.com

Binary Tree



log2base2.com

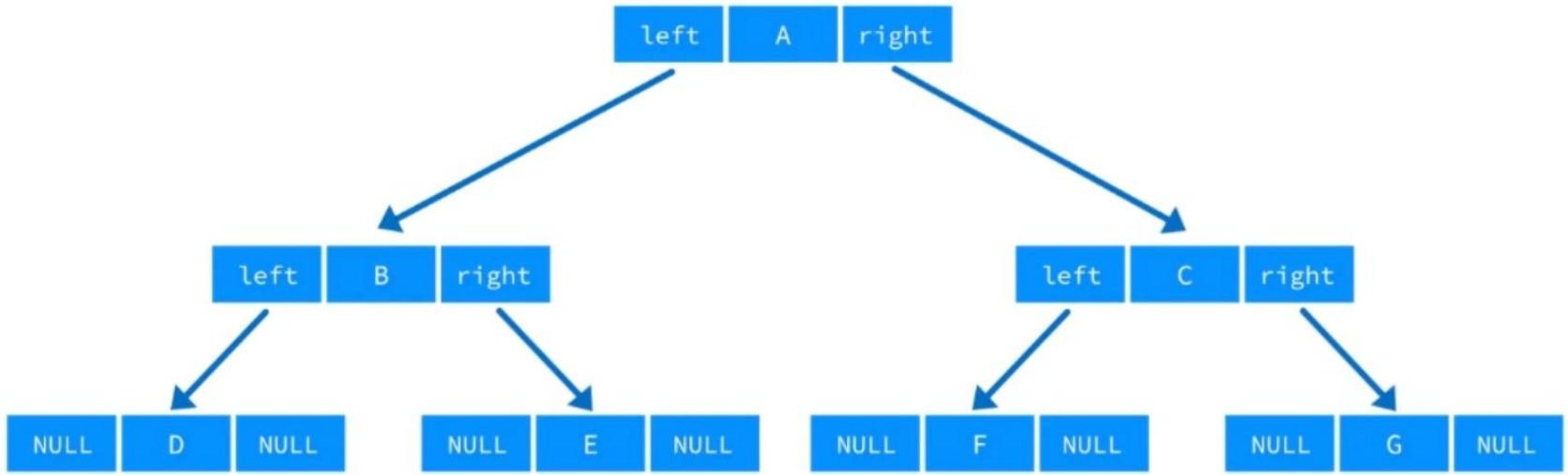
Binary Tree



log2base2.com

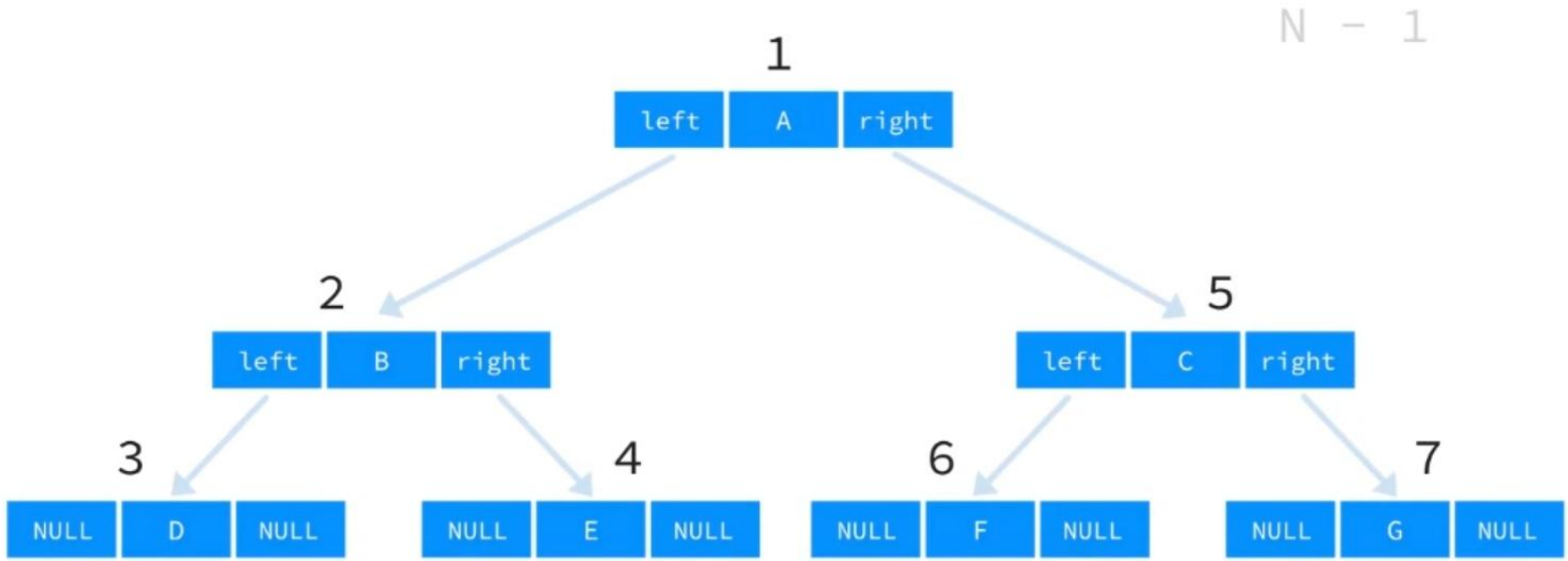
Binary Tree

N - 1



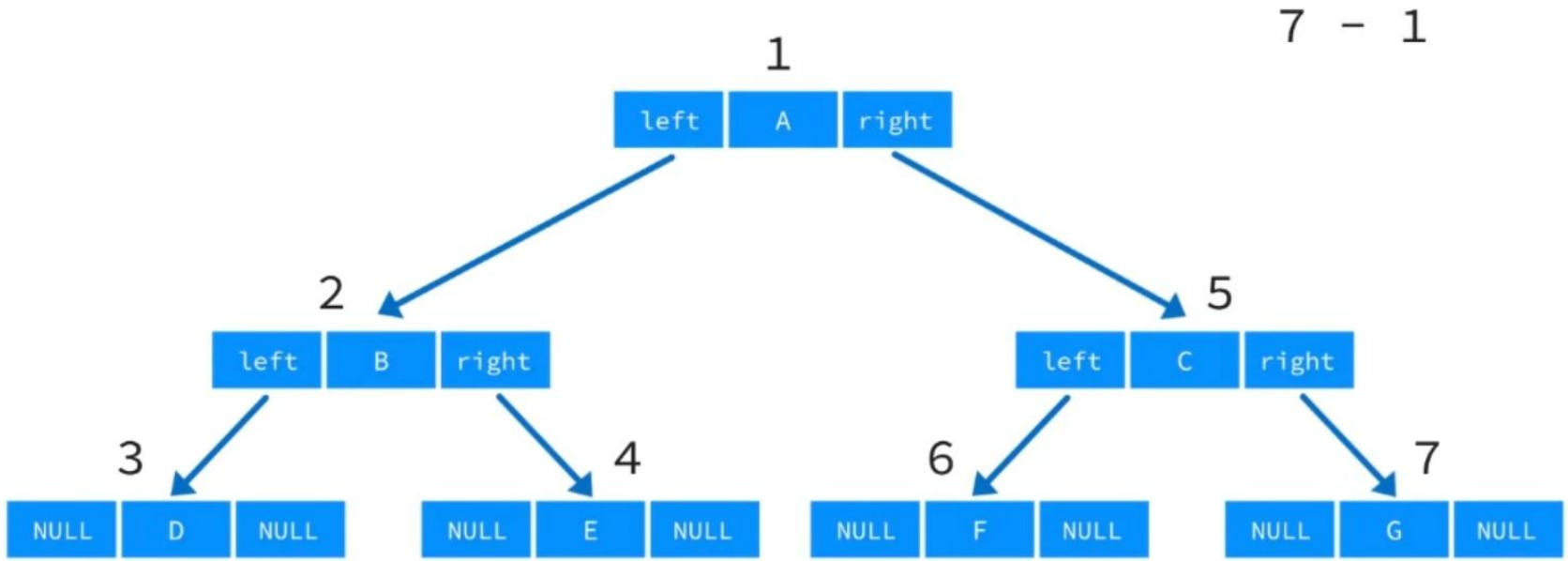
log2base2.com

Binary Tree



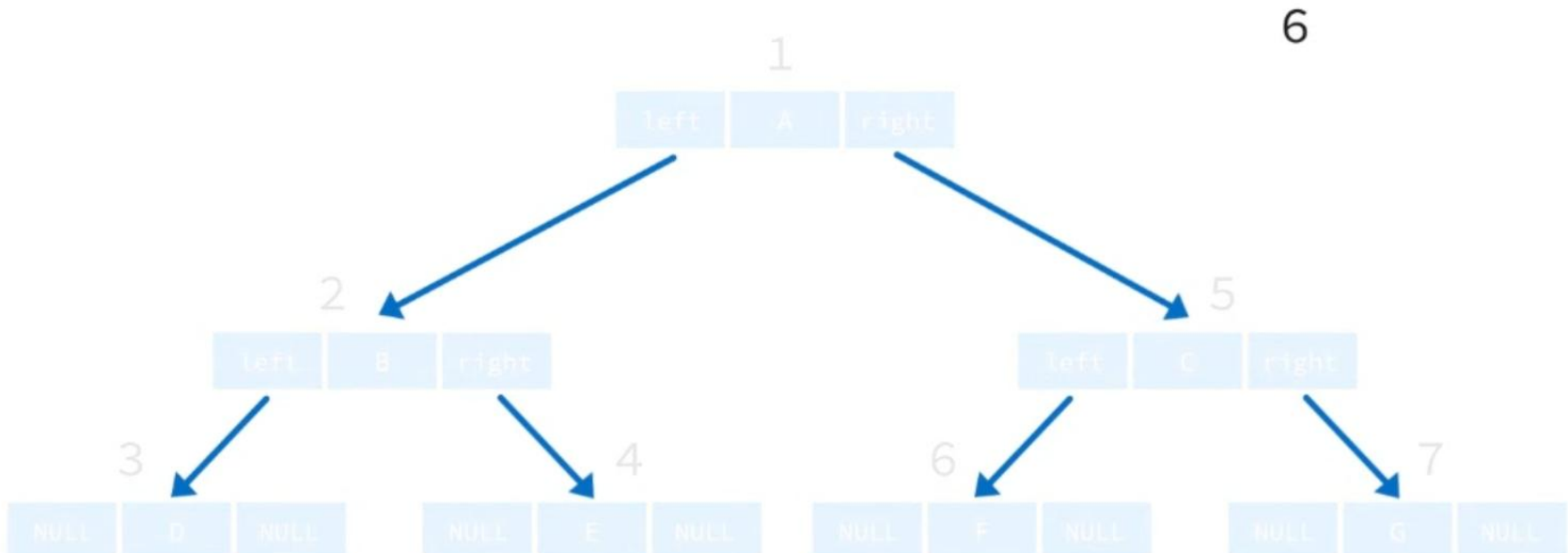
log2base2.com

Binary Tree



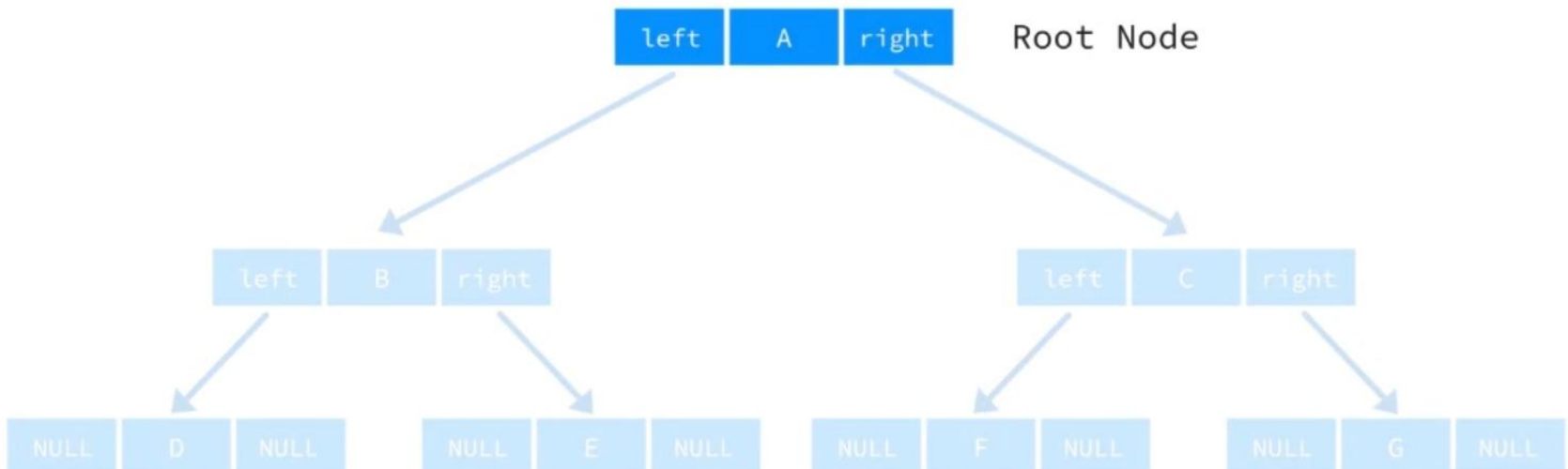
log2base2.com

Binary Tree



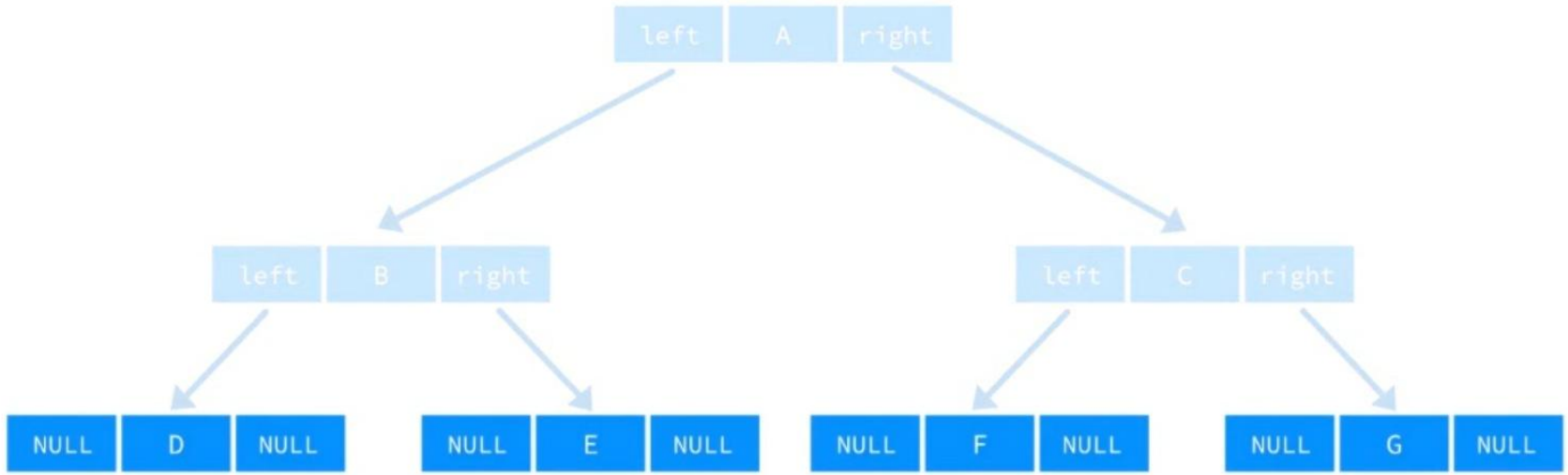
log2base2.com

Binary Tree



log2base2.com

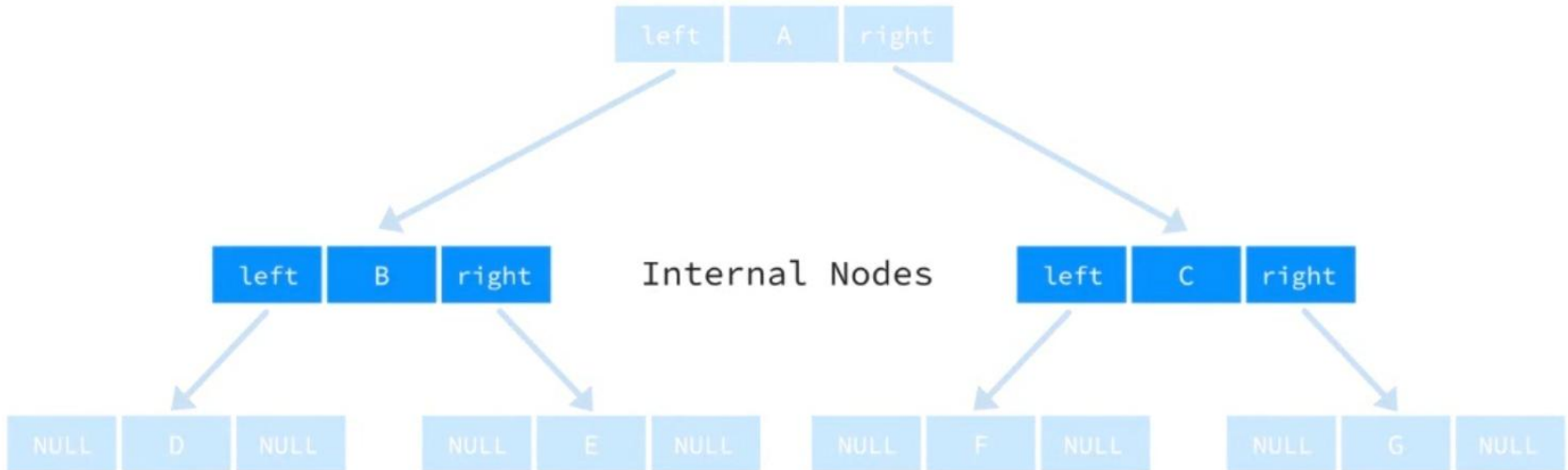
Binary Tree



Leaf Nodes

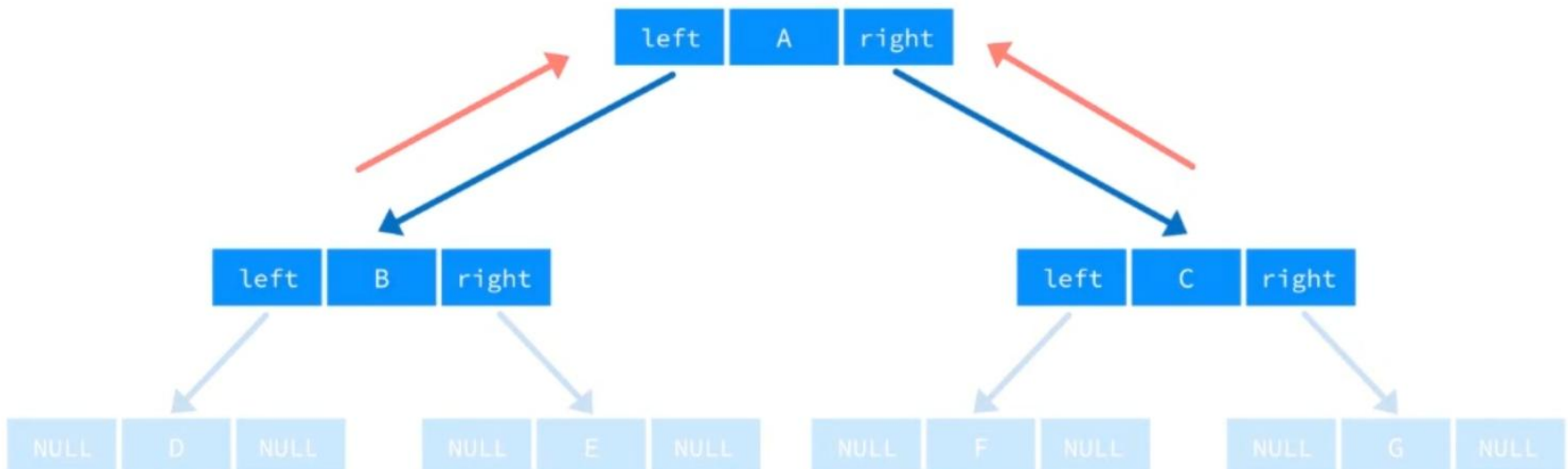
log2base2.com

Binary Tree



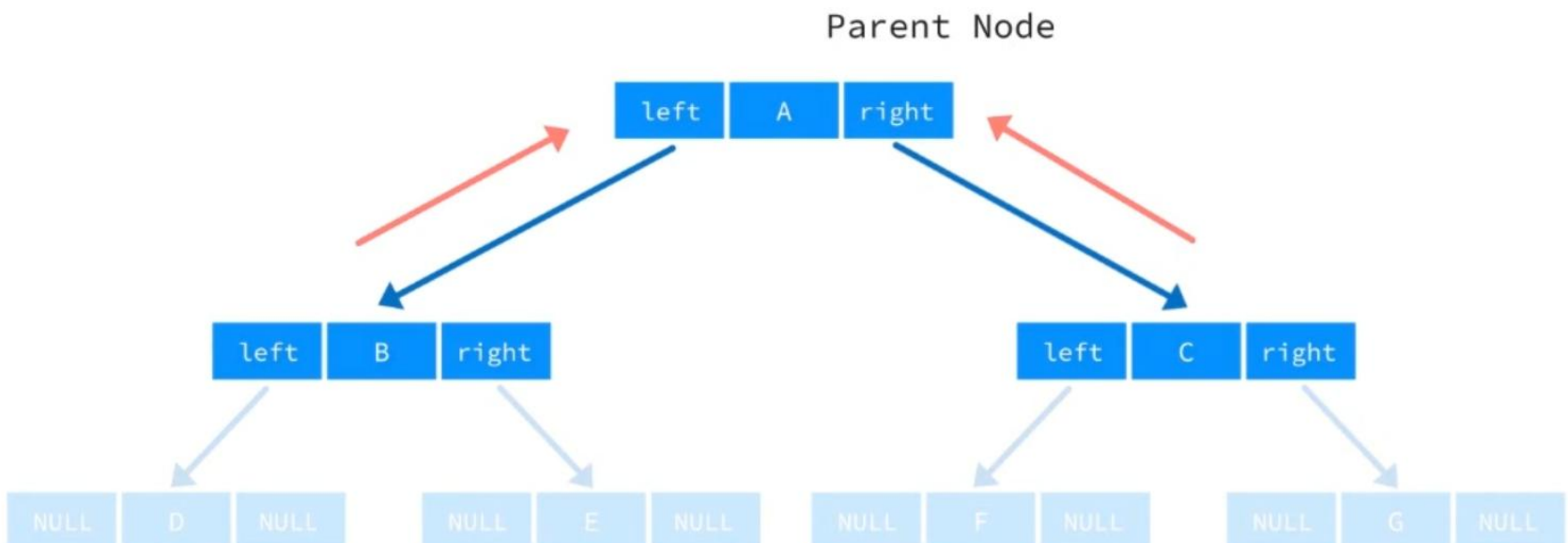
log2base2.com

Binary Tree



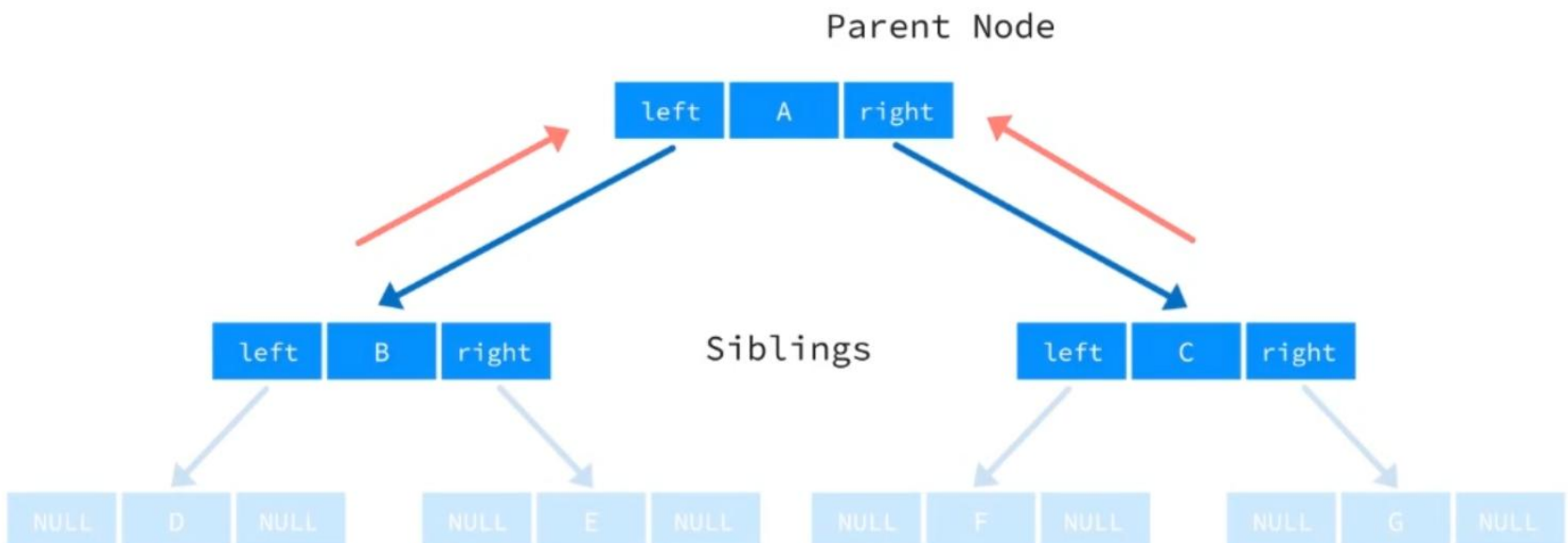
log2base2.com

Binary Tree



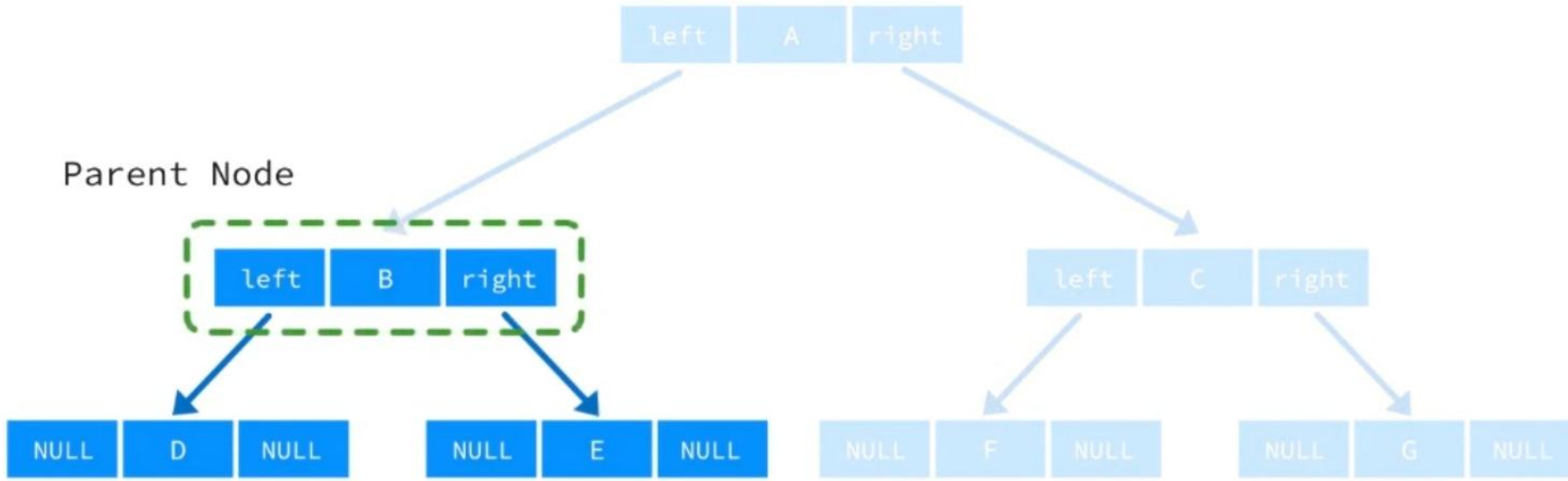
log2base2.com

Binary Tree



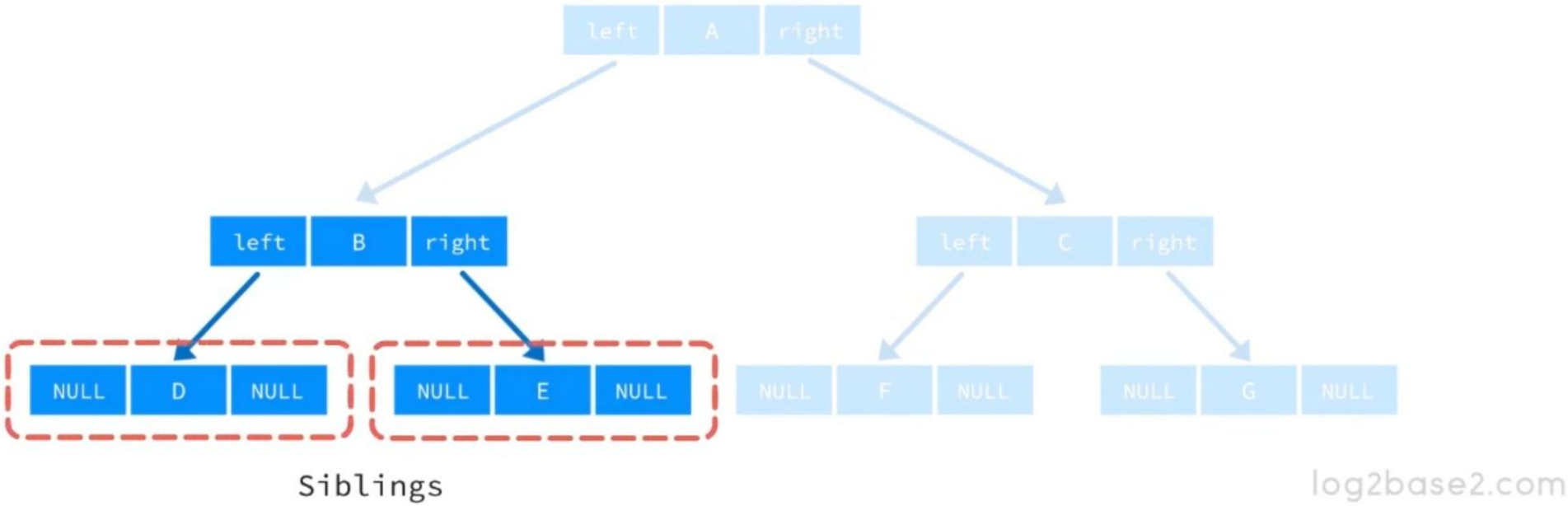
log2base2.com

Binary Tree

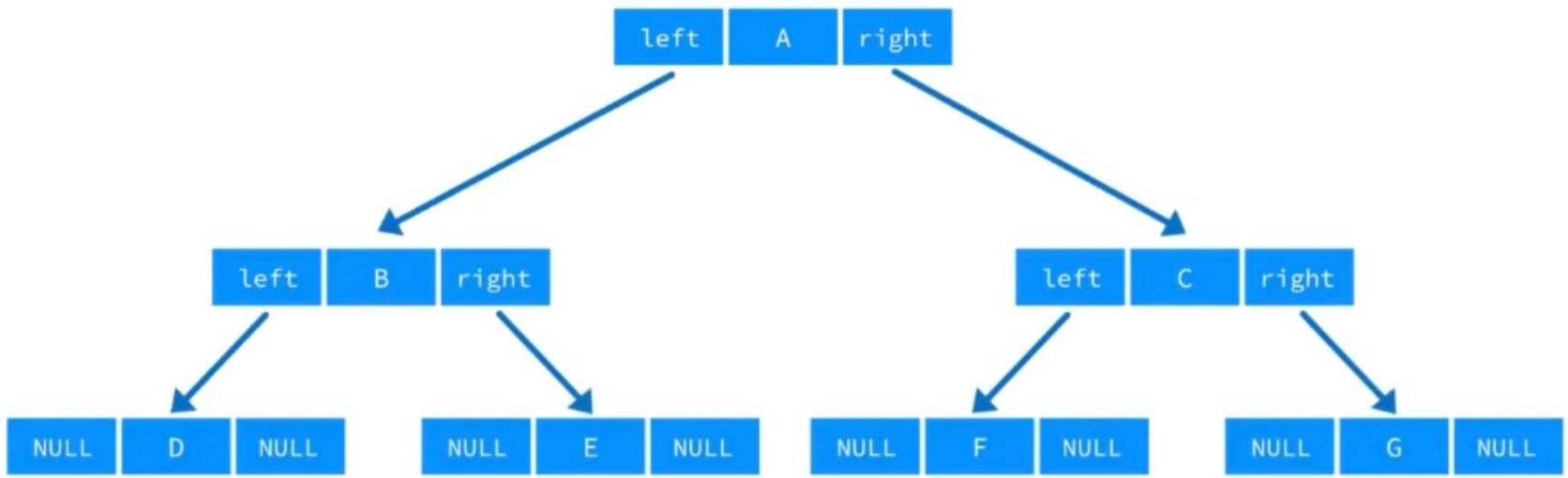


log2base2.com

Binary Tree

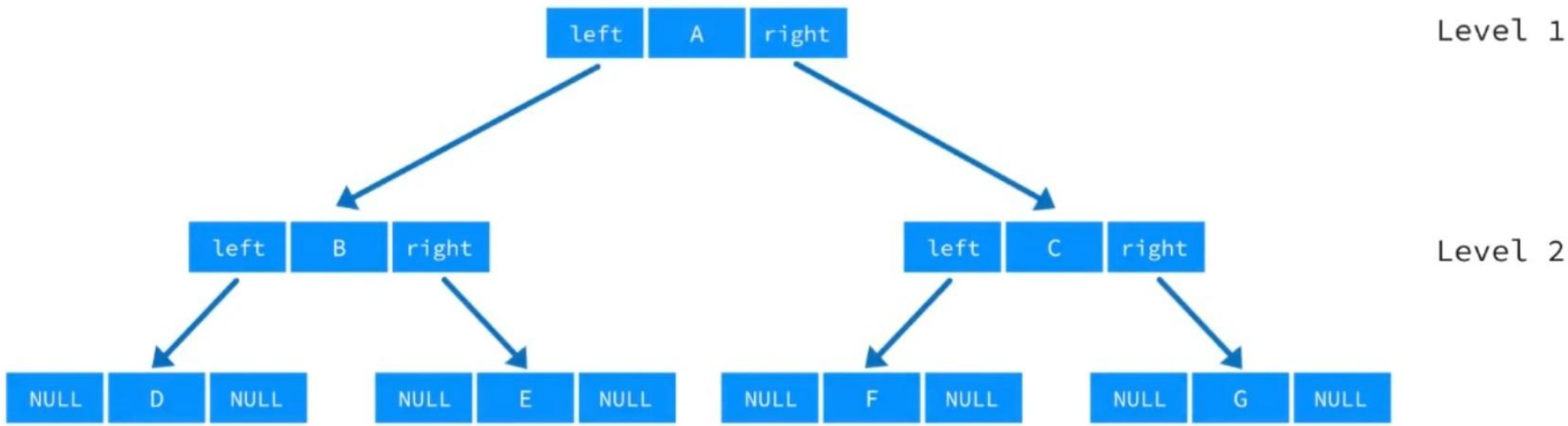

left
A
right
left
B
right
left
C
right
NULL
D
NULL
NULL
E
NULL
NULL
F
NULL
NULL
G
NULL
Siblings
log2base2.com

Binary Tree



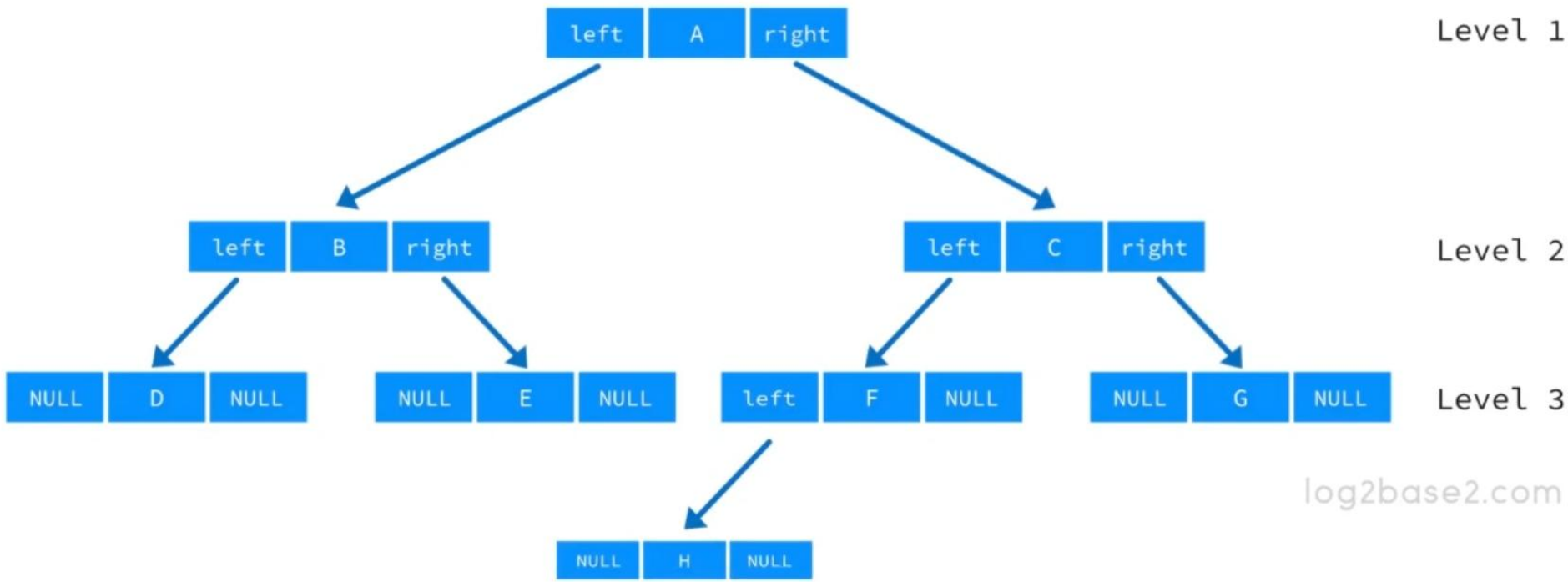
log2base2.com

Binary Tree

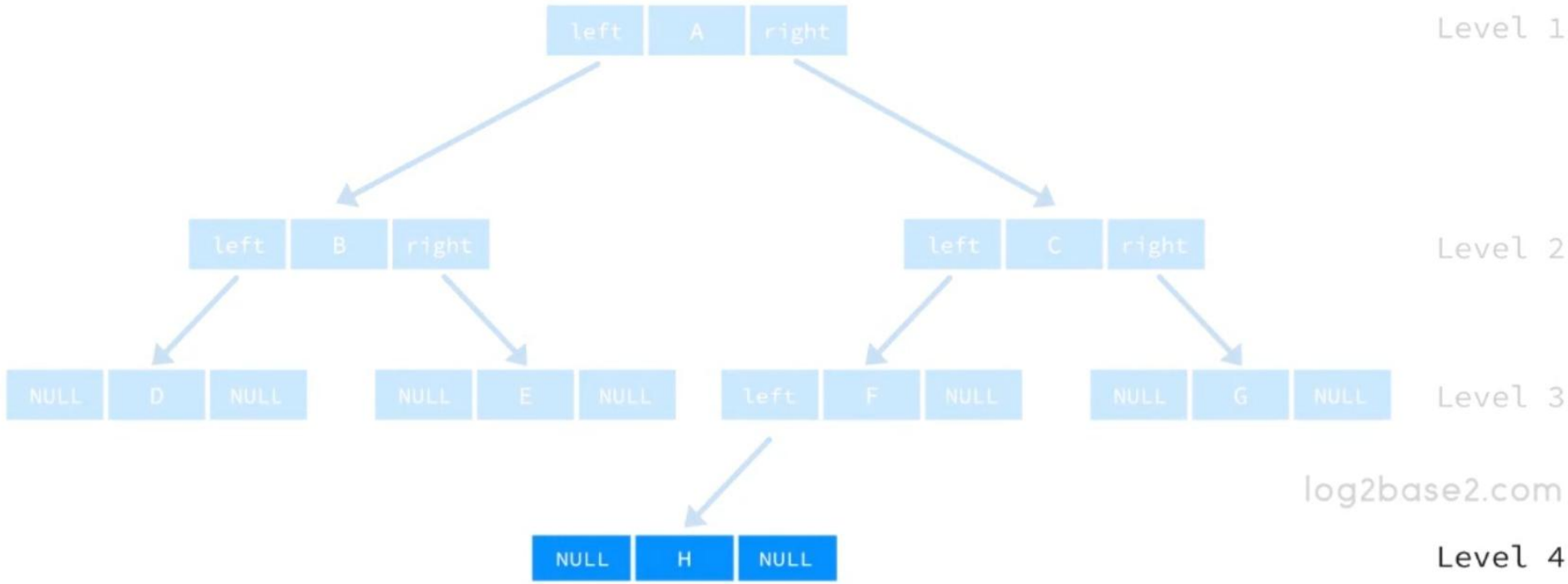


log2base2.com

Binary Tree


left
A
right
Level 1
left
B
right
left
C
right
Level 2
NULL
D
NULL
NULL
E
NULL
left
F
NULL
NULL
G
NULL
Level 3
NULL
H
NULL
log2base2.com

Binary Tree


left
A
right
Level 1
left
B
right
left
C
right
Level 2
NULL
D
NULL
NULL
E
NULL
left
F
NULL
NULL
G
NULL
Level 3
log2base2.com
NULL
H
NULL
Level 4